



Programming Fundamentals

By

Dr Suhail Asghar

Dawood University Of Engineering &
Technology Karachi

Arrays

- Arrays are one dimensional, two-dimensional etc.
- Consecutive/contiguous group of memory locations
- Same name and type (int, char, float etc.)
 - Student_Names[50],
Student_GPA[50],
Dept_Names[10]
- Compiler reserves a memory section to hold all of data
 - The lowest address corresponds to the first element
- To refer to an element
 - Specify array name and position number (index)
 - Format: arrayname [position number]
 - First element at position 0 (index 0)
 - Last element at position N-1 (index N-1)
 - 10-element array c
 - c[0], c[1] ... c[9]

Declaring Arrays

- When declaring arrays, specify
 1. Name
 2. Type of an array
 - Any data type
 3. Number of elements/Size of an array
- `type arrayName[ arraySize ];`
 - `int c[ 10 ]; // array of 10 integers`
 - `float d[ 3284 ]; //`
- Declaring multiple arrays of same type
 - Use comma separated list, like regular variables
 - `int b[ 100 ], x[ 27 ];`

Initializing arrays

- For loop
 - Set each element
- Initializer list
 - Specify each element when array declared

```
int n[ 5 ] = { 1, 2, 3, 4, 5 };
```

 - If not enough initializers, rightmost elements 0
 - If too many syntax error
- To assign value to every element individually

```
int A[3];  
A [0] = 2; A [1] = 12;  
A[2] = 1;
```
- If array size omitted, initializers determine size
- `int n[] = { 1, 2, 3, 4, 5 };`
- 5 initializers, therefore 5 element array

Examples Using Arrays

- **Accessing Array elements**

- Values of any of elements in an array can be accessed

- Arrayname [index]

- n[0], n[1], student_name[5], vowel[3], course_name[1] etc.

- Variable1=vowel[3]; #include <iostream>

- x=n[0]+n[1]; using namespace std;

```
int num_array [] = {15, 30, 75, 40, 120};
```

```
int i, result=0;
```

```
int main ()
```

```
{
```

```
    for ( i=0 ; i<5 ; i++ )
```

```
    {
```

```
        result += num_array[i];
```

```
    }
```

```
    cout << result;
```

```
    return 0;    }
```

Examples Using Arrays

- Array size
 - Can be specified with constant variable (**const**)
 - **const int size = 20;**
 - Constants cannot be changed
 - Constants must be initialized when declared
 - Also called named constants or read-only variables

while a C++ program `cin` takes
the number of elements as input,
stores the user-entered values in
an array, and then prints the

array elements

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "How many
elements do you want to
enter? ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n
<< " elements:\n";
```

```
    for(int i = 0; i < n;
i++) {
        cin >> arr[i];
    }
    cout << "You
entered:\n";
    for(int i = 0; i < n;
i++) {
        cout << arr[i] <<
" ";
    }
    return 0;
}
```

Accessing Array elements

- Values of any of elements in an array can be accessed
- Arrayname [index]
 - n[0], n[1], student_name[5], vowel[3], course_name[1] etc.
 - Variable1=vowel[3];
 - x=n[0]+n[1];

```
#include <iostream>
using namespace std;
int num_array [] = {15, 30, 75, 40, -120};
int i, result=0;
int main () {
    for ( i=0 ; i<5 ; i++ )
    {
        result += num_array[i];
    }
    cout << result;
    return 0;
}
```


Write a C++ program that
prints a horizontal histogram
of the array

```
#include <iostream>
using namespace std;
int main() {
    int var = 10;
    int arr[var] = {19, 3, 15, 7,
11, 9, 13, 5, 17, 1};
    for (int i = 0; i < var; i++) {
        for (int j = 0; j < arr[i];
j++) {
            cout << "*";
        }
        cout << endl;
    }
    return 0;
}
```

Write a C++ program that
prints a horizontal histogram
of the array (all input)

```
#include <iostream>
using namespace std;
int main() {
    int var;
    cout << "How many
parameters are
there" << endl;
    cin >> var;
    int arr[var];
    cout << "Enter values
corresponding to the
variables" << endl;
    for (int i = 0; i < var;
i++)
        cin >> arr[i];
    for (int i = 0; i <
var; i++) {
        for (int j = 0; j
< arr[i]; j++) {
            cout << "*";
        }
        cout << endl;
    }
    return 0;
}
```

Examples Using Arrays

```
1
2
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include <iomanip>
9
10 using std::setw;
11
12 int main()
13 {
14     int n[ 10 ]; // n is an array of 10 integers
15
16     // initialize elements of array n to 0
17     for ( int i = 0; i < 10; i++ )
18         n[ i ] = 0; // set element at location i to 0
19
20     cout << "Element" << setw( 13 ) << "Value" << endl;
21
22     // output contents of array n in tabular format
23     for ( int j = 0; j < 10; j++ )
24         cout << setw( 7 ) << j << setw( 13 ) << n[ j ] << endl;
```

Declare a 10-element array of integers.

Initialize array to 0 using a for loop. Note that the array has elements `n[0]` to `n[9]`.

Examples Using Arrays

```
26     return 0; // indicates successful termination
27
28 } // end main
```

Element	Value
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

Write a C++ program that takes an array of numbers as input and sorts it in ascending order.

```
#include <iostream>
using namespace std;
int main()
{
    const int ARRAY_SIZE=5;
    int array1[ARRAY_SIZE], i ,j ,temp;
    cout<<"Program for Sorting an Array in Asending Order\n";
    // Input Array Elements
    for (i = 0; i < ARRAY_SIZE; i++)
    {
        cout<<"Enter the Number : "<< i <<"    : ";
        cin>>array1[i];    }
    cout<<endl;
    // Display Elements of Unsorted Array
    cout<<"UnSorted Array:  ";
    for (i = 0; i < ARRAY_SIZE; ++i)
        cout<<array1[i]<<"    ";
    cout<<endl;
```

```
// Array Sorting - Asending Order
for (i = 0; i < ARRAY_SIZE; ++i)
{
    for (j = i + 1; j < ARRAY_SIZE; ++j)
    {
        if (array1[i] > array1[j])
        {
            temp = array1[i];
            array1[i] = array1[j];
            array1[j] = temp;
        }
    }
}
```

```
cout<<"Sorted Array (Asending Order): ";
for (i = 0; i < ARRAY_SIZE; ++i)
    cout<<array1[i]<<" ";
```

```
return 0;
}
```

```
Program for Sorting an Array in Asending Order
Enter the Number : 0 : 9
Enter the Number : 1 : 3
Enter the Number : 2 : 5
Enter the Number : 3 : 7
Enter the Number : 4 : 1
UnSorted Array: 9 3 5 7 1
Sorted Array (Asending Order): 1 3 5 7 9
```

Multi-dimensional Array

- Two-dimensional array (2D) represents information arranged in rows and columns (i.e., subscripts)
- To identify a particular table element, we must specify two subscripts—by convention, the first identifies the element's **row** and the second identifies the element's **column**
- Arrays with two or more dimensions are known as **multidimensional** arrays and can have more than two dimensions

Multi-dimensional Array

- “Array of arrays”
- In general, an array with m rows and n columns is called an *m -by- n* array
- e.g. 2D array with 3 rows & 4 columns (3x4) array
- an element name of the form $a[i][j]$

	Column 0	Column 1	Column 2	Column 3
Row 0	$a[0][0]$	$a[0][1]$	$a[0][2]$	$a[0][3]$
Row 1	$a[1][0]$	$a[1][1]$	$a[1][2]$	$a[1][3]$
Row 2	$a[2][0]$	$a[2][1]$	$a[2][2]$	$a[2][3]$

Diagram illustrating the structure of a 2D array (3x4) with row and column subscripts. The array is represented as a grid of elements. The row subscript (i) and column subscript (j) are shown for each element. The array name (a) is also indicated.

Labels and arrows pointing to the subscripts in the element $a[2][1]$:

- array name (points to a)
- Row subscript (points to 2)
- Column subscript (points to 1)

Multi-dimensional Array

- To initialize

- Default of 0

- Initializers grouped by row in braces

```
int b[ 2 ][ 2 ] = { { 1, 2 }, { 3, 4 } };
                   Row 0   Row 1
```

1	2
3	4

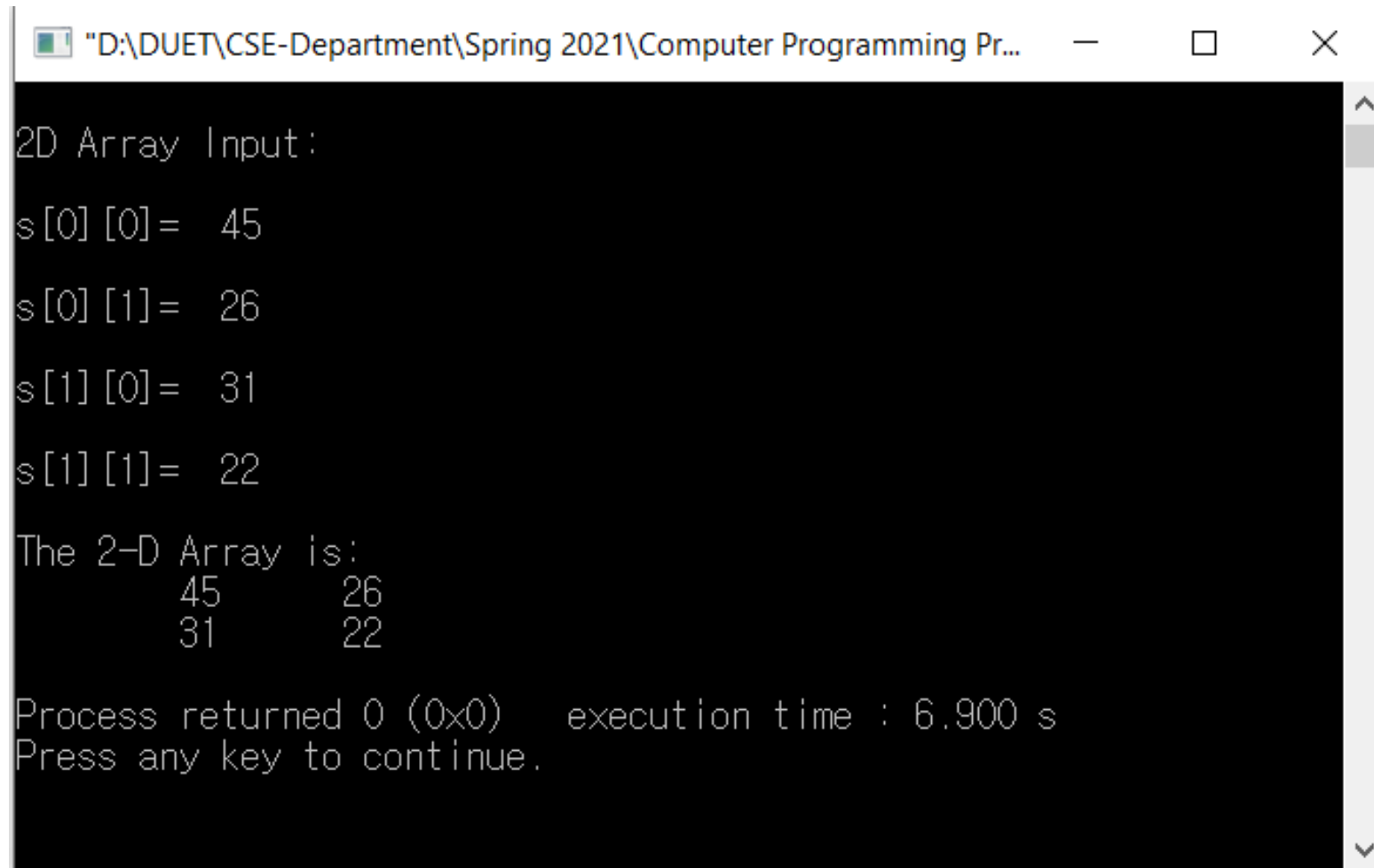
```
int b[ 2 ][ 2 ] = { { 1 }, { 3, 4 } };
```

1	0
3	4

Multi-dimensional Array

```
#include<iostream>
using namespace std;
int main( ) {
    int  s[2][2];
    int  i, j;
    cout<<"\n2D Array Input:\n";
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            cout<<"\ns["<<i<<"]["<<j<<"]=" ";
            cin>>s[i][j];
        }
    }
    cout<<"\nThe 2-D Array is:\n";
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            cout<<"\t"<<s[i][j];
        }
        cout<<endl;
    }
    return 0;
}
```

Multi-dimensional Array



A screenshot of a Windows command prompt window. The title bar shows the file path "D:\DUET\CSE-Department\Spring 2021\Computer Programming Pr...". The window contains the following text:

```
2D Array Input:  
s[0][0]= 45  
s[0][1]= 26  
s[1][0]= 31  
s[1][1]= 22  
  
The 2-D Array is:  
    45    26  
    31    22  
  
Process returned 0 (0x0)   execution time : 6.900 s  
Press any key to continue.
```

Write a C++ program that takes marks of students in multiple courses, stores them in a 2D array, calculates each student's average, and displays the results.

```
#include <iostream>
using namespace std;
int main() {
    const int students = 3;
    const int courses = 3;
    float marks[students][courses], average[students];
    for (int i = 0; i < students; i++) {
        float total = 0;
        cout << "Enter marks for Student " << i + 1 << ":\n";
        for (int j = 0; j < courses; j++) {
            cin >> marks[i][j];
            total += marks[i][j];
        }
        average[i] = total / courses;
    }
    cout << "\nAverage marks of students:\n";
    for (int i = 0; i < students; i++) {
        cout << "Student " << i + 1 << ": " << average[i]
        << "\n";
    }
    return 0;}
```

Array Container (STL)

- Array can be created using array template
 - **array <type, size> arrayname;**
 - E.g. **array <int, 10> array_using_template**
- The elements of an array also can be initialized in the array declaration by list of initializers
 - **array< int, 5 > n = { 32, 27, 64, 18, 95 };**
- If there are fewer initializers than array elements, the remaining array elements are initialized to zero
 - **array <int, 5>n= {3, 4, 5}**
- If the array size and an initializer list are specified in an array declaration, the number of initializers must be less than or equal to the array size
 - **array <int, 5>n= {3, 4, 5, 6, 7, 8}**

Array Container (STL)

```
#include <iostream>
#include<iomanip>
#include<array>

using namespace std;

int main()
{
    array<int, 5>n={10, 50, 40, 35, 15};
    cout<<"Element"<< setw(13) << "Value"<<endl;
    for (int i=0; i<5; i++)
        cout<<setw(7)<<i<<setw(13)<<n[i]<<endl;
    return 0;
}
```

Array Container (STL)

```
#include <iostream>
#include <array>

using namespace std;

int main ()
{
    array<int,10> array1={15,20,33,24,55,60,74,83,92,100};
    for (int i=0; i<10;i++){
        //at(n) function returns the value at that index
        cout <<"The array element at inedx " <<i<<": " <<
            array1.at(i)<<endl;
    }
    return 0;
}
```

```
The array element at inedx 0: 15
The array element at inedx 1: 20
The array element at inedx 2: 33
The array element at inedx 3: 24
The array element at inedx 4: 55
The array element at inedx 5: 60
The array element at inedx 6: 74
The array element at inedx 7: 83
The array element at inedx 8: 92
The array element at inedx 9: 100
```

Write a C++ program that uses `std::array` and `sort()` to sort an integer array and a character array, then prints each sorted element with its index.

```
#include <iostream>
#include <array>
#include <algorithm> // contains sort and binary_search
using namespace std;
int main () {
    array<int,10> array1 = {150,200,303,24,55,60,87,38,15,100};
    sort(array1.begin(), array1.end());
    cout<<"First Array Sorting"<<endl;
    for (int i=0; i<10;i++){
        //at(n) function returns the value at that index
        cout <<"The array element at index"<<i<<": "<<array1.at(i)<<endl;
    }
    cout<<"Second Array Sorting"<<endl;
    array<char,6> array2={'R','A','M','E','S','H'};
    sort(array2.begin(), array2.end());
    for (int j=0; j<6;j++){
        //at(n) function returns the value at that index
        cout <<"The array element at index"<<j<<": "<<array2.at(j)<<endl;
    }
    return 0;
}
```


Frist Array Sorting

The array element at inedx 0: 15
The array element at inedx 1: 24
The array element at inedx 2: 38
The array element at inedx 3: 55
The array element at inedx 4: 60
The array element at inedx 5: 87
The array element at inedx 6: 100
The array element at inedx 7: 150
The array element at inedx 8: 200
The array element at inedx 9: 303

Second Array Sorting

The array element at inedx 0: A
The array element at inedx 1: E
The array element at inedx 2: H
The array element at inedx 3: M
The array element at inedx 4: R
The array element at inedx 5: S

in a `std::array`, displays them unsorted, sorts them alphabetically, prints the sorted list, and uses `binary_search` to check for specific colors.

```
#include <array>
#include <string>
#include <algorithm> // contains sort and binary_search
using namespace std;
int main()
{
    const size_t arraySize = 7; // size of array colors
    array< string, arraySize > colors = { "red", "orange",
    "yellow", "green", "blue", "indigo", "violet" };
    // output original array
    cout << "Unsorted array: \n";
    for ( string color : colors )
        cout << color << " ";
    sort(colors.begin(), colors.end()); //sort contents of colors
    // output sorted array
    cout << "\nSorted array: \n";
    for ( string item : colors )
        cout << item << " ";
```

```
// search for "indigo" in colors
bool found= binary_search(colors.begin(),colors.end(),"indigo");
cout << "\n\n\"indigo\" " << ( found ? "was" : "was not" )<< "
found in colors" << endl;
// search for "cyan" in colors
found = binary_search( colors.begin(), colors.end(), "cyan" );
cout << "\"cyan\" " << ( found ? "was" : "was not" )<< " found in
colors" << endl;
return 0;
}
```

Unsorted array:

red orange yellow green blue indigo violet

Sorted array:

blue green indigo orange red violet yellow

"indigo" was found in colors

"cyan" was not found in colors

Dynamically Allocated Array

- Static array limitations: (Compile-time size)
 - Must specify size first as a constant
 - Array elements are stored on the stack
 - Very limited application, as in many cases the number of elements may not be known until the program runs
- May use partially filled arrays for more flexibility
 - Must estimate maximum size needed
 - Wastes memory
- **Dynamic arrays: (Run-time size)**
 - Can grow and shrink as needed
 - Implemented as a pointer to a dynamically allocated array
 - Array elements are stored on the heap

Dynamically Allocated Array

STL template **vector**

- STL class template `vector` is similar to class template `array`, but also supports dynamic resizing
 - Dynamically grow to accommodate more elements
- Standard class template `vector` is defined in header `<vector>` and belongs to namespace `std`.
- `vectors` can be defined to store most data types
- `vector` member function `size` obtain the number of elements in the vector
- Vector are implemented as dynamic arrays with `list interface`
`vector<string>nameVec;    vector<int>Marks;`
`vector<double>GPAVec;`

Dynamically Allocated Array STL template vector

```
#include<iostream>
#include<vector>
using namespace std;
int main(){
vector<int> v; //Vector Implementation
// Inserting Values in Vector
v.push_back(1); v.push_back(2); v.push_back(3);
v.push_back(4); v.push_back(5);
cout<<"Size of vector Before Removal="<<v.size() << "\n";
// Output Values of vector
for (int i=0; i<v.size(); i++)
    cout << v[i] << " ";
v.erase(v.begin() + 2); // Remove 3rd element
cout<<"\nSize of vector After removal=" <<v.size()<< "\n";
// Output Values of vector
for (auto it : v) cout << it << " ";
v.insert(v.begin() + 2 , 7); // insert element at 3rd Location
cout << "\nSize of vector After Insert=" << v.size() << "\n";
    for (auto it : v)        cout << it << " ";
return 0;}
```

```
Size of vector Before Removal=5
1 2 3 4 5
Size of vector After removal=4
1 2 4 5
Size of vector After Insert=5
1 2 7 4 5
```

Dynamically Allocated Array STL template vector

```
#include<iostream>
#include<vector>
using namespace std;
int main()
{
    vector<int> vec {2,4,6,8};
    vector<int> vec2 {1,3,5,7};
    cout<<"Initially first vector: ";
    for(auto i=vec.begin(); i<vec.end(); i++)
    {
        cout<<" "<<*i;
    }
    cout<<"\nInitially second vector: ";
    for(auto i=vec2.begin(); i<vec2.end(); i++)
    {
        cout<<" "<<*i;
    }
    //Inserting vec2 at the beginning of the vec vector
    vec.insert(vec.begin(),vec2.begin(),vec2.end());
    cout<<"\n\nThe modified vector is: ";
    for(auto i=vec.begin(); i<vec.end(); i++)
    {
        cout<<" "<<*i;
    }
    return 0;
}
```

Initially first vector: 2 4 6 8

Initially second vector: 1 3 5 7

The modified vector is: 1 3 5 7 2 4 6 8